

Минитендер.рф — платформа строительных закупок

Строительные закупки без посредников. Отправил список материалов — AI распознал, нашёл поставщиков, разослал RFQ, собрал КП в конкурентный лист. Переписку с поставщиками ведёт LLM в деловом стиле.

Быстрый старт (5 минут)

python start.py # backend :8000 + frontend :3000

Демо-вход: dev@test.com / test12345. Открой http://localhost:3000

Требования: Python 3.13 + uv, Node.js 20+.

Основной сценарий

- Заказчик создаёт заявку списком материалов (мастер из 3 шагов).
- MaterialIntel + LLM-парсер** разбирает позиции: категория, material_type, количество, confidence; по неполным позициям задаёт уточняющие вопросы.
- Заказчик указывает точку доставки (карта или текст).
- Матчер** скорит поставщиков (категории, расстояние, рейтинг, ассортимент, тип, модерация); при нехватке — **автоdiscovery** ищет новых в интернете.
- RFQ-рассылка: текст письма генерирует **LLM (llm_writer)** строго по фактам заявки; при сбое — статичный шаблон.
- Поставщик заполняет КП по ссылке /quote/{token} или просто отвечает письмом — inbound-парсер извлекает цены из текста письма.
- Заказчик получает уведомления о каждом КП и видит **конкурентный лист** с лучшим предложением.

Технологический стек

Слой	Технология
Бэкенд	Django 5 + DRF + Celery
Фронтенд	Next.js 15 + TypeScript + Tailwind
База данных	PostgreSQL 16 + PostGIS (dev: SQLite)
Кэш/очереди	Redis (опционально, sync fallback в dev)
ИИ	DeepSeek API (парсинг, материал-интеллект, переписка)
Карты	2GIS + Yandex Geocoder

Слой	Технология
Поиск компаний	DaData API
Прокси/контейнеры	Nginx, Docker Compose

□ Ключевые возможности

Парсинг заявок (MaterialIntel)

- **Универсальный LLM-промт** — понимает любые формулировки, включая опечатки («бурсчатка») и латиницу («bruschatka»); матрица из 20 реальных формулировок — 100% точности (см. docs/QA_PARSE_MATRIX.md)
- **Оценка полноты** — confidence 0.0–1.0 + уточняющие вопросы по неполным позициям (хранятся в `RequestItem.clarification_question`, показываются в UI)
- **Whitelist категорий/единиц** + JSON Schema валидация ответов LLM
- **Идемпотентность** — повторный parse делает diff-update, не создаёт дубли
- **Кэш материал-профилей** (`MaterialProfile`) — синонимы и поисковые запросы по нормализованному названию

Поиск поставщиков

- **Скоринг**: категории (50) + расстояние (30) + рейтинг (10) + полнота профиля (10) + производитель (5) + material_type (15) + ассортимент (20); раскрываемый `score_breakdown` в UI
- **Product-match rule** — поставщик без товара в ассортименте отсекается
- **Модерация (B4)**: `rejected` исключён из подбора, `unverified` — понижающий коэффициент ×0.9 и бейдж «На проверке»; staff-кнопки в `/lk/suppliers`, `bulk-verify`
- **Автоdiscovery** — при <5 совпадениях поиск новых поставщиков в интернете, счётчик «найдено N новых» в UI
- **Ручное добавление (B5)** — форма в `/lk/suppliers`, автообогащение каталога + геокодинг

Переписка с поставщиками (LLM, B9)

- **Библиотека промтов** (`apps/emails/prompts.py`): приглашение, напоминания 24ч/2ч, уточняющий вопрос, ответ на вопрос, благодарность за КП, победа/отказ
- **Жёсткие правила безопасности**: только факты заявки, запрет обещаний («гарантируем», «скидка», «оплатим») — пост-проверка, нарушения → `needs_review`, письмо НЕ отправляется
- **Fallback** — при недоступности LLM уходит статичный шаблон
- **Eval-набор** — 21 эталонный тест (`backend/tests/test_llm_writer.py`)

Цикл КП

- **Публичная страница КП** `/quote/{token}` — без авторизации, throttle 30/мин, валидация цен
- **Inbound email (B1)** — ответ письмом на `rfq-XXX@in.минитендер.рф` создаёт КП; LLM извлекает цены из текста (`inbound_parser`), вопросы поставщика → авто-ответ из `llm_writer`; транспорт: IMAP-polling (`fetch_inbound`) или вебхуки Mailgun/ generic
- **Напоминания (B10)** — `send_deadline_reminders` за 24ч и 2ч до дедлайна, идемпотентно
- **Уведомления заказчику (B7)** — письмо о каждом КП и о готовности конкурентного листа

Асинхронность

- **5 Celery-задач** — `parse`, `match`, `send_rfq`, `geocode`, `discover`
- **Sync fallback** — в dev всё выполняется синхронно, без Redis

□ Тесты и QA

```
cd backend && uv run --extra dev python -m pytest tests/ -q # 78 тестов
cd frontend && node node_modules/next/dist/bin/next build # чистая сборка
```

- 78 автотестов: позитивные сценарии + 27 негативных (A2) + 21 eval LLM-писем; внешние API в тестах не вызываются
- `docs/QA_SECURITY.md` — аудит безопасности (IDOR/XSS закрыты кодом)
- `docs/QA_LOAD.md` — нагрузочный sanity (p95 создания 0.75с, подбора 0.47с)
- `docs/QA_PARSE_MATRIX.md` — матрица парсинга граничных формулировок

□ Структура проекта

```
backend/
  apps/
    accounts/      # auth, JWT, геокодинг
    requests/      # заявки, парсер, матчер, discovery, MaterialIntel
    suppliers/     # поставщики, модерация, каталог
    quotes/        # КП, приглашения, конкурентный лист, публичный API
    emails/        # шаблоны, llm_writer, inbound, напоминания
  scripts/        # seed, parse_matrix, load_sanity, dedupe_suppliers
  tests/          # 78 автотестов
frontend/
  app/lk/          # личный кабинет: заявки, мастер, поставщики
  app/quote/       # публичная страница КП
docs/              # QA-отчёты, API, OPERATIONS, INBOUND_SETUP
deploy/           # prod-контур (docker-compose.prod.yml, nginx)
```

Документация: `docs/API.md`, `docs/OPERATIONS.md`, `docs/INBOUND_SETUP.md`.

□ Контрибьютинг

1. Форкни репозиторий
2. Создай ветку `feat/твоя-фича`
3. Убедись, что pytest зелёный и фронт собирается
4. Открой Pull Request

☐ Лицензия

MIT © Минитендер.рф